

به نام خدا

گزارش پروژه پایانی

طراحی و شبیه سازی پردازنده ۶ بیتی با VHDL

اعضا گروه:

مجتبی خالقی

محمدامین محمدی نژاد

چکیده

در این پروژه یک پردازنده ساده ۶ بیتی با چهار رجیستر عمومی، شمارنده برنامه، ثبات دستور، حافظه ۶۴ کلمه‌ای و واحد محاسبه و منطق طراحی و شبیه سازی شده است. در مرحله نخست دستورهای پایه ADD، SUB، LOAD و JNZ پیاده سازی شدند. در مرحله دوم عمل ضرب به صورت نرم افزاری و با استفاده از جمع تکراری، کاهش شمارنده و پرش شرطی انجام شد. در مرحله سوم دستور جدید MUL به مجموعه دستورهای افزوده شد و کنترل حلقه ضرب از برنامه اسمبلی به داخل واحد کنترل منتقل گردید. در مرحله تکمیلی نیز یک اسمبلر با Python ساخته شد که برنامه اسمبلی را به کلمات ۶ بیتی مناسب حافظه پردازنده تبدیل می کند. نتیجه شبیه سازی نمونه ۸×۶ برابر ۴۸ (HEX 30) بوده است.

مقدمه و هدف پروژه

هدف پروژه آشنایی عملی با اجزای اصلی یک پردازنده، مسیر داده، واحد کنترل، قالب دستور، اجرای چندچرخه‌ای دستورها و ارتباط میان نرم‌افزار و سخت‌افزار است. پردازنده طراحی شده عمداً کوچک و ۶ بیتی است تا رفتار داخلی آن در شکل موج شبیه‌سازی به‌وضوح قابل مشاهده باشد.

- عرض داده، رجیسترها، گذرگاه و حافظه: ۶ بیت
- تعداد رجیسترهای عمومی: چهار رجیستر R0 تا R3
- ظرفیت حافظه: ۶۴ کلمه ۶ بیتی
- روش طراحی واحد کنترل: ماشین حالت محدود چندچرخه‌ای
- زبان توصیف سخت‌افزار: VHDL

ساختار کلی پردازنده

معماری پردازنده از اجزای زیر تشکیل شده است:

- چهار ثبات عمومی ۶ بیتی با خروجی پرچم صفر
- شمارنده برنامه ۶ بیتی برای آدرس‌دهی حافظه
- ثبات دستور ۶ بیتی برای نگهداری دستور جاری
- حافظه ۶۴×۶ با رابط جداگانه برنامه‌ریزی توسط Testbench
- دو مالتی‌پلکسر برای انتخاب رجیسترهای Rx و Ry
- ALU با دو عمل جمع و تفریق
- واحد کنترل برای تولید سیگنال‌های بارگذاری، افزایش PC، فرمان ALU و انتخاب گذرگاه

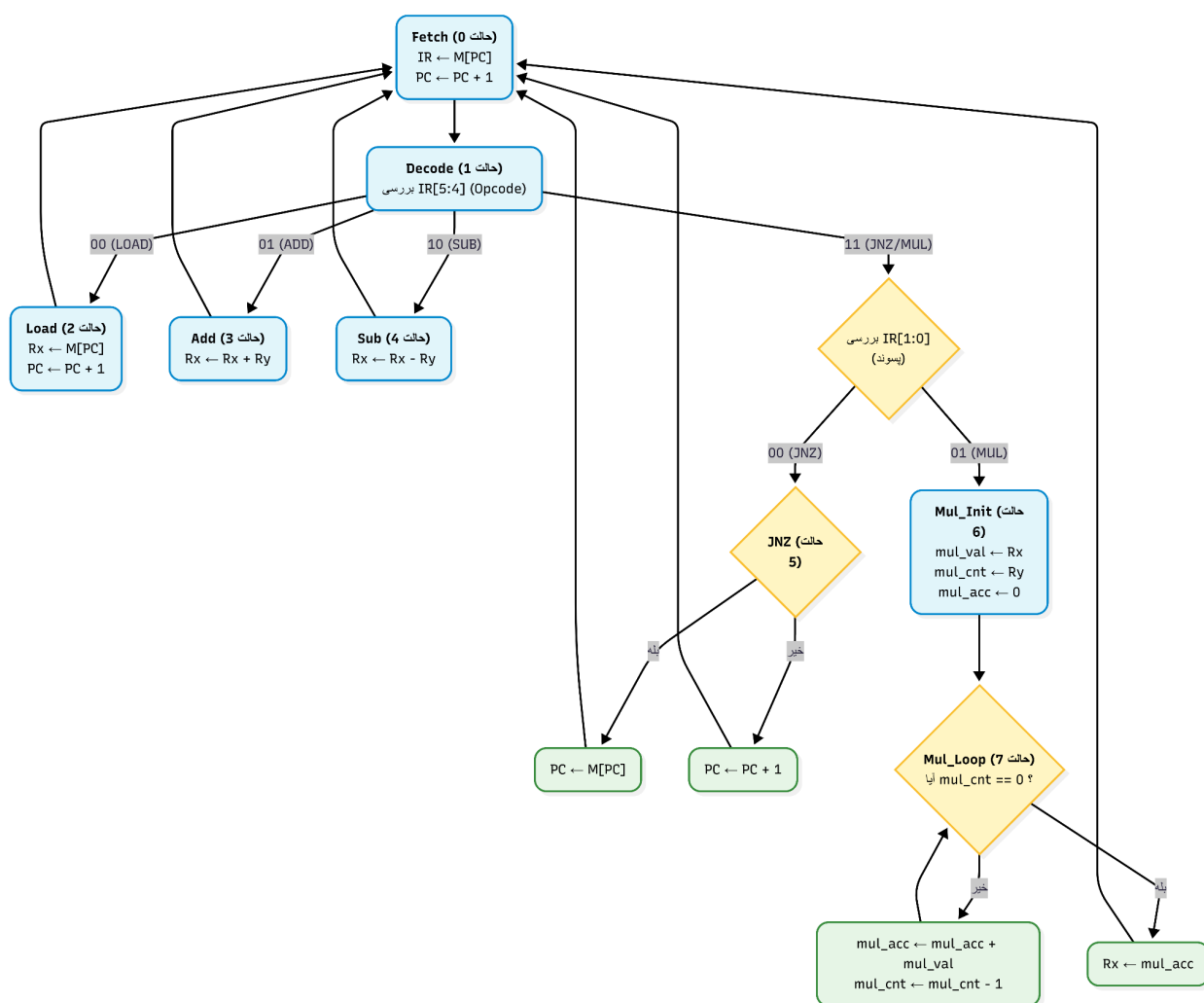
معماری پردازنده و مجموعه دستورات (ISA)

این پردازنده دارای معماری ۶-بیتی است و از یک شمارنده برنامه (PC) شش بیتی، چهار ثبات همه‌منظوره (R0 تا R3)، یک باس مشترک و یک ALU تشکیل شده است.

کد اسمبلی	کد ماشین (باینری)	عملیات (RTL)	توضیحات
LOAD Rx, VALUE	Rx 00 00 [VALUE]	$Rx \leftarrow M[PC]$ $PC \leftarrow PC + 1$	بارگذاری مقدار ۶-بیتی در ثبات Rx
ADD Rx, Ry	Rx Ry 01	$Rx \leftarrow Rx + Ry$	جمع مقادیر دو ثبات و ذخیره در Rx
SUB Rx, Ry	Rx Ry 10	$Rx \leftarrow Rx - Ry$	تفریق مقدار Ry از Rx
JNZ Rx, ADDR	Rx 00 11 [ADDR]	If (Rx != 0) $PC \leftarrow$ [M[PC]	پرش شرطی در صورت ناصفر بودن Rx
MUL Rx, Ry	Rx 01 11 Ry 00 00	$Rx \leftarrow Rx * Ry$	ضرب سخت‌افزاری چند چرخه‌ای (بخش ۳)

واحد کنترل و ماشین حالت الگوریتمی (ASM)

واحد کنترل بر اساس یک ماشین حالت متناهی (FSM) کار می‌کند که شامل فازهای واکنشی (Fetch)، رمزگشایی (Decode) و اجرا (Execute) است. در بخش سوم، حالت‌های مربوط به ضرب سخت‌افزاری (st_mul_init و st_mul_loop) به این ماشین حالت اضافه شدند.



نتایج شبیه‌سازی و صحت‌سنجی

در این بخش، صحت عملکرد معماری طراحی شده از طریق شبیه‌سازی با نرم‌افزار Active-HDL اثبات می‌شود. سیگنال‌های کلیدی مانند وضعیت ماشین حالت (state_debug)، مقادیر ثبات‌ها (r0_debug و r1_debug) و شمارنده‌ها برای هر بخش بررسی شده است.

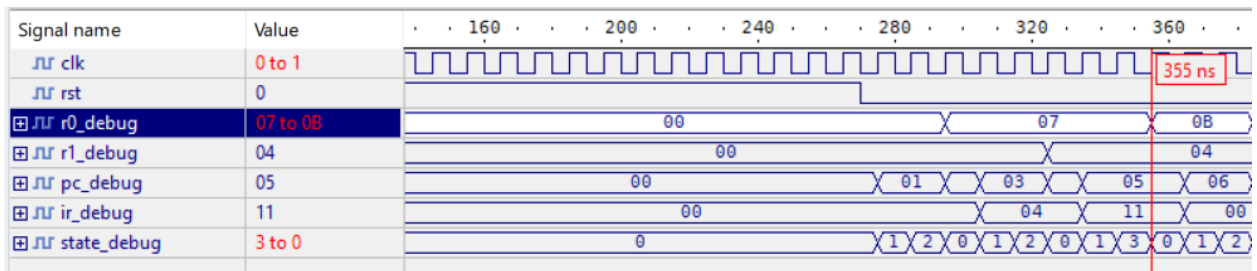
پیاده‌سازی دستورهای پایه

در بخش اول واحد کنترل شامل حالت‌های JNZ، FETCH، DECODE، LOAD، ADD، SUB بود. در حالت FETCH دستور از حافظه وارد IR و شمارنده برنامه یک واحد افزایش داده می‌شد. سپس در DECODE بر اساس Opcode، حالت اجرای مناسب انتخاب می‌گردید.

```
FETCH → DECODE → LOAD → FETCH
FETCH → DECODE → ADD → FETCH
FETCH → DECODE → SUB → FETCH
FETCH → DECODE → JNZ → FETCH
```

شبیه‌سازی بخش اول: پردازنده پایه (تست ۷ + ۴)

در این شبیه‌سازی، مقادیر ۷ و ۴ در ثبات‌های R0 و R1 بارگذاری شده و سپس دستور ADD اجرا می‌شود. با توجه به شکل موج، مقدار ثبات R0 در نهایت به ۱۱ (باینری ۰۰۱۰۱۱ یا هگز ۰۸) تغییر می‌یابد که نشان‌دهنده عملکرد صحیح ALU و واحد کنترل است.



ضرب نرم افزاری

در بخش دوم هیچ دستور MUL در سخت افزار وجود نداشت. بنابراین ضرب با یک برنامه اسمبلی و با ترکیب دستورهایی موجود انجام شد. پردازنده از ماهیت عملیات ضرب اطلاع نداشت و فقط دستورهایی ADD، SUB و JNZ را طبق روال عادی اجرا می کرد.

```
LOAD R0, 0    ; حاصل
LOAD R1, 6     ; عددی که تکراری جمع می شود
LOAD R2, 8     ; شمارنده
LOAD R3, 1     ; مقدار کاهش شمارنده

LOOP:
ADD R0, R1
SUB R2, R3
JNZ R2, LOOP
```

در این برنامه R0 حاصل، R1 عدد ۶، R2 شمارنده ۸ و R3 مقدار ثابت ۱ را نگه می دارد. در هر دور حلقه، عدد ۶ به حاصل اضافه و شمارنده یک واحد کم می شود. حلقه تا صفر شدن R2 ادامه پیدا می کند.

شبیه سازی بخش دوم: ضرب نرم افزاری (۶ × ۸)

این شبیه سازی اجرای یک حلقه نرم افزاری (شامل ADD و JNZ) را نشان می دهد. ثابت R0 نقش انباشتگر را دارد و در هر چرخه مقدار آن ۶ واحد افزایش می یابد. همزمان ثابت R2 به عنوان شمارنده کاهش می یابد. در پایان شبیه سازی، مقدار R0 به ۴۸ (هگز 30) می رسد.

Signal name	Value	240	320	400	480	560	640
clk	0 to 1						
rst	0						
r0_debug	10 to 1E	00	06	0C	12	18	
r2_debug	04	00	08	07	06	05	04
pc_debug	09	05	07	09	0A	0B	09
ir_debug	11	04	08	0C	11	2B	38
state_debug	3 to 0						

افزودن دستور سخت‌افزاری MUL

در بخش سوم دستور MUL به مجموعه دستورهای پردازنده اضافه شد. از دید برنامه‌نویس، ضرب با یک دستور انجام می‌شود؛ ولی داخل پردازنده همچنان از الگوریتم جمع تکراری و یک اجرای چندچرخه‌ای استفاده شده است. این روش نسبت به ساخت یک ضرب‌کننده ترکیبی کامل، سربار سخت‌افزاری کمتری دارد.

LOAD R0, 8

LOAD R1, 6

MUL R0, R1

برای اجرای MUL سه ثابت داخلی به CPU اضافه شد:

ثبات داخلی	وظیفه
mul_acc	نگهداری حاصل موقت ضرب
mul_value	نگهداری مقدار Rx برای جمع تکراری
mul_count	نگهداری تعداد جمع‌های باقی‌مانده و مقدار اولیه Ry

واحد کنترل نیز با حالت‌های MUL_INIT و MUL_LOOP توسعه یافت. در MUL_INIT مقادیر اولیه در ثابت‌های داخلی قرار می‌گیرند. در MUL_LOOP تا زمانی که mul_count صفر نشده است، mul_value به mul_acc افزوده و شمارنده یک واحد کم می‌شود. پس از صفر شدن شمارنده، حاصل از طریق گذرگاه به رجیستر مقصد نوشته می‌شود.

$mul_acc \leftarrow 0$

$mul_value \leftarrow R[Rx]$

$mul_count \leftarrow R[Ry]$

while mul_count $\neq$ 0:

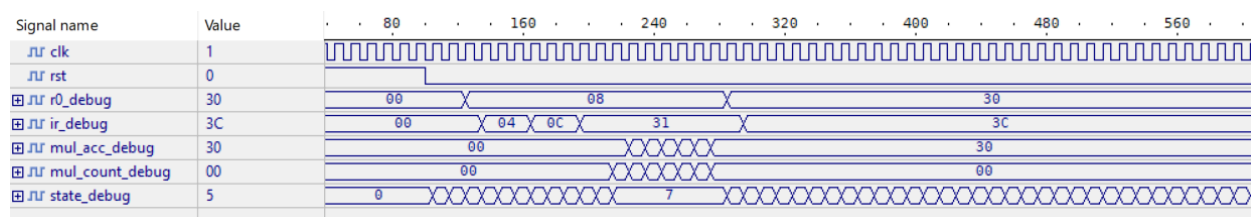
$mul_acc \leftarrow mul_acc + mul_value$

$mul_count \leftarrow mul_count - 1$

$R[Rx] \leftarrow mul_acc$

شبیه‌سازی بخش سوم: ضرب سخت‌افزاری (۶ × ۸)

با ارتقاء سخت‌افزار، ضرب به صورت چند چرخه‌ای و مستقل از کدهای اسمبلی طولانی انجام می‌شود. سیگنال state_debug ورود به حالت‌های شماره ۶ و ۷ را نشان می‌دهد. انباشتگر سخت‌افزاری (mul_acc_debug) با سرعت بالا مقدار ۴۸ (هگز ۳۰) را محاسبه کرده و در رجیستر مقصد می‌نویسد.



شکل موج MUL؛ تغییر mul_acc از ۰ تا ۴۸ و mul_count از ۶ تا صفر

تفاوت دقیق بخش دوم و بخش سوم

هر دو بخش حاصل یکسانی تولید می‌کنند، اما محل کنترل الگوریتم ضرب متفاوت است. در بخش دوم حلقه ضرب داخل برنامه اسمبلی است؛ در بخش سوم حلقه داخل واحد کنترل و مسیر داده پردازنده قرار دارد.

بخش سوم: دستور سخت‌افزاری MUL	بخش دوم: ضرب نرم‌افزاری	ویژگی
وجود دارد	وجود ندارد	وجود دستور MUL
داخل Control Unit	داخل برنامه اسمبلی	محل حلقه ضرب
ثبات داخلی mul_count و حالت MUL_LOOP	دستورهای SUB و JNZ	کنترل تکرار
معمولاً فقط Rx و Ry	R0، R1، R2 و R3	رجیسترهای عمومی مصرفی
کم	زیاد	تعداد دستورهای برنامه
برای MUL فقط یک بار	برای هر دور چند بار	تعداد Fetch و Decode
کمتر	بیشتر	پیچیدگی نرم‌افزار
بیشتر	کمتر	پیچیدگی سخت‌افزار
۴۸	۴۸	حاصل نمونه

بنابراین تفاوت اصلی در پاسخ عددی نیست؛ تفاوت در این است که چه بخشی مسئول مدیریت جمع‌های تکراری است. در بخش دوم برنامه‌نویس باید حلقه را بنویسد، اما در بخش سوم برنامه‌نویس فقط MUL را صادر می‌کند و پردازنده مراحل داخلی را انجام می‌دهد.

پیاده‌سازی اسمبلر با زبان پایتون

هدف از این بخش پیاده‌سازی یک مفسر (Assembler) با استفاده از زبان‌های سطح بالا (پایتون) است تا کدهای نوشته شده به زبان اسمبلی (مانند `LOAD R0, 7`) را به صورت خودکار به کدهای باینری ۶-بیتی قابل فهم برای پردازنده (Machine Code) تبدیل کند.

نحوه عملکرد و معماری اسمبلر: اسمبلر توسعه‌یافته در یک اسکریپت پایتون (`Assembler.py`) پیاده‌سازی شده است. این برنامه فایل متنی حاوی کدهای اسمبلی را خط به خط می‌خواند و پس از پردازش، فایل باینری نهایی (`program.txt`) را برای بارگذاری در حافظه‌ی پردازنده (RAM) تولید می‌کند.

ورودی اسمبلر فایل `code.asm` و خروجی آن فایل `program.txt` است. هر خط فایل خروجی یک کلمه ۶ بیتی از حافظه را نمایش می‌دهد.

code.asm



assembler.py



program.txt



tb_cpu_part4.vhd



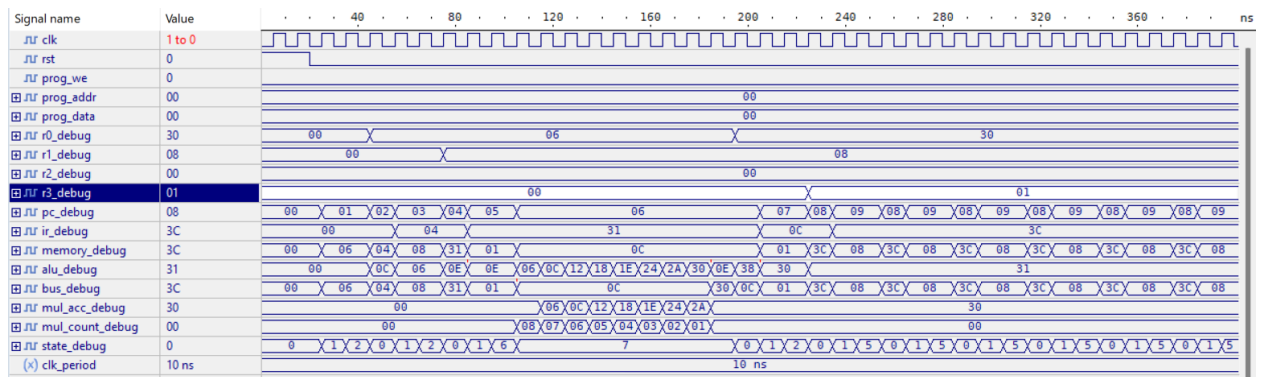
Memory → CPU

در گذر اول، آدرس Labelها محاسبه می‌شود. در گذر دوم، دستورها بر اساس جدول Opcode تبدیل شده و آدرس Labelها در دستورهای JNZ جایگزین می‌شوند. اسمبلر همچنین رجیستر نامعتبر، عدد خارج از محدوده صفر تا ۶۳، دستور ناشناخته، Label تکراری و برنامه بزرگ‌تر از ۶۴ کلمه را تشخیص می‌دهد.

```
python assembler.py program.asm -o program.txt
```

خروجی برنامه نمونه ۶×۸ به صورت زیر است:

```
000000
001000
000100
000110
001100
000001
110001
000001
111100
001000
```



جمع‌بندی

در این پروژه یک پردازنده ۶ بیتی چندچرخه‌ای طراحی و شبیه‌سازی شد. ابتدا مجموعه دستورهای پایه پیاده‌سازی گردید. سپس ضرب با استفاده از نرم‌افزار و حلقه‌ای شامل ADD، SUB و JNZ انجام شد. در ادامه دستور سخت‌افزاری MUL به پردازنده افزوده شد و مسئولیت کنترل ضرب به واحد کنترل منتقل گردید. با استفاده مجدد از جمع‌کننده ALU، سربار سخت‌افزاری محدود نگه داشته شد. در نهایت اسمبلر Python امکان تبدیل خودکار برنامه اسمبلی به فایل حافظه را فراهم کرد. نتیجه نمونه ۸×۶ در هر دو روش برابر ۴۸ بود، اما بخش سوم برنامه کوتاه‌تر و مجموعه دستورهای پردازنده کامل‌تری ارائه داد.

فایل‌های تحویلی

- فایل‌های اجزای پایه: alu6.vhd، mux4_6.vhd، pc6.vhd، register6.vhd و memory6.vhd
- نسخه نهایی CPU.vhd و control_unit.vhd شامل دستور MUL
- Testbench بخش دوم برای ضرب نرم‌افزاری
- Testbench بخش سوم برای دستور سخت‌افزاری MUL
- code.asm، assembler.py و program.txt
- tb_cpu_part4.vhd برای خواندن خروجی اسمبلر
- گزارش حاضر به همراه تصاویر شکل موج و خروجی Console